LESSON 03 · GIT · 50 MIN

3교시 원격 저장소와 협업 — push가 거절되는 이유

push 거절, fetch와 pull의 차이는 세 저장소의 위치 관계 하나로 전부 설명됩니다. 동료가 먼저 push한 상황을 직접 만들어서 거절을 재현하고, 올바른 순서로 해결해 봅니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

세 곳의 저장소와 이동 방향



origin/main 은 원격이 아니라 내 컴퓨터에 적힌 메모입니다. '내가 마지막으로 확인했을 때 원격은 여기까지였다'는 기록이죠. fetch는 이 메모만 갱신하고, pull은 메모를 갱신한 뒤 내 브랜치에 합치기까지 합니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`git clone <주소>`	원격 저장소를 기록째로 복제	프로젝트를 처음 받을 때
`git remote -v`	연결된 원격 주소 확인	어디로 push되는지 확인할 때
`git push -u origin main`	로컬 커밋을 원격으로 올림	작업을 공유할 때
`git fetch`	원격 변화를 받아만 옴 (합치지 않음)	합치기 전에 확인하고 싶을 때
`git pull`	fetch + merge	최신 상태로 맞출 때
`git log --oneline origin/main..main`	아직 push 안 한 내 커밋 목록	push 전에 점검할 때
`Pull Request`	합치자는 제안 + 리뷰 + 기록	팀 작업의 기본 단위
`git tag -a v1.0.0 -m "..."`	특정 커밋에 이름표 붙이기	릴리즈 시점을 표시할 때

HANDS-ON

실습 순서

STEP 01 GitHub 저장소와 인증 준비

원격에 올릴 빈 저장소를 만들고 인증 방식을 정합니다.

```
# GitHub 로그인 → 우측 상단 + → New repository
# 이름: git-practice · Public · README 체크는 해제한 채 생성
# HTTPS 방식이면 Settings → Developer settings → Personal access tokens 에서 토큰 발급
```

확인 포인트

- [ ] 저장소 주소(https://github.com/아이디/git-practice.git)를 복사해 두었다
- [ ] 토큰을 발급했다면 안전한 곳에 저장했다 (다시 볼 수 없습니다)

2021년부터 **비밀번호로는 push할 수 없습니다.** HTTPS를 쓰면 비밀번호 자리에 토큰을 넣어야 하고, SSH를 쓰면 키를 등록해야 합니다.

STEP 02 원격 연결하고 첫 push

로컬 저장소와 GitHub을 연결합니다.

```
git remote add origin https://github.com/아이디/git-practice.git
git remote -v
git branch -M main
git push -u origin main
```

확인 포인트

- [ ] remote -v 에 fetch와 push 두 줄이 보인다
- [ ] GitHub 페이지를 새로고침하면 파일과 커밋 기록이 보인다

-u 는 이 브랜치의 기본 원격을 기억하라는 뜻입니다. 다음부터는 **git push** 만 쳐도 됩니다.

STEP 03 동료 역할 만들기 (clone)

다른 사람이 같은 저장소를 받은 상황을 만듭니다.

```
cd ..
git clone https://github.com/아이디/git-practice.git teammate
cd teammate
git log --oneline
git remote -v
```

확인 포인트

- [ ] 커밋 기록까지 그대로 복제되었다
- [ ] clone은 remote 연결까지 자동으로 되어 있다

> 2인 1조라면 한 명은 원본 폴더, 한 명은 clone 폴더를 맡으세요. 혼자라면 두 폴더를 오가며 진행하면 됩니다.

STEP 04 동료가 먼저 push하기

원격이 나보다 앞서 있는 상황을 만듭니다.

```
# teammate 폴더에서
echo "동료의 작업" > t.txt
git add . && git commit -m "동료 작업 추가"
git push
```

확인 포인트

- [ ] GitHub에 t.txt 가 보인다

STEP 05 fetch와 pull의 차이 관찰 — 오늘의 핵심

fetch가 내 파일을 바꾸지 않는다는 것을 눈으로 확인합니다.

```
# 원본 git-practice 폴더로 돌아와서
git fetch
git status
ls
git log --oneline --graph --all
git pull
ls
```

확인 포인트

- [ ] fetch 후 status에 Your branch is behind 'origin/main' by 1 commit 이라고 나온다
- [ ] 그런데 ls 에는 t.txt 가 없다 — fetch는 파일을 건드리지 않는다
- [ ] pull 을 하고 나서야 t.txt 가 생긴다

> 이 차이가 이번 시간의 핵심입니다. **fetch는 정보만 가져오고, pull은 그 정보를 내 브랜치에 반영합니다.** 남의 변경을 먼저 확인하고 합치고 싶을 때 fetch를 씁니다.

STEP 06 push 거절 재현하고 해결하기

non-fast-forward 거절을 일부러 만들어 올바른 순서를 익힙니다.

```
# teammate 폴더에서
echo "동료 두 번째" >> t.txt && git add . && git commit -m "동료 2" && git push
# 원본 폴더에서 (pull 하지 않은 채로)
echo "내 작업" > y.txt && git add . && git commit -m "내 작업"
git push
git pull
git push
```

확인 포인트

- [ ] 첫 push가 rejected · non-fast-forward 로 거절된다
- [ ] pull 하면 병합 커밋이 만들어진다 (에디터가 열리면 그대로 저장)
- [ ] 두 번째 push는 성공한다

> Git이 막는 이유는 **동료의 커밋이 사라지는 것을 방지하기 위해서**입니다. 해결 순서는 언제나 **pull → (충돌이 있으면 해결) → push** 입니다. -f 로 강제 push하는 것은 팀 작업에서 매우 위험합니다.

STEP 07 Pull Request 만들고 병합하기

실무의 기본 협업 단위를 경험합니다.

```
git switch -c feature-readme
echo "# git-practice" > README.md
git add . && git commit -m "README 추가"
git push -u origin feature-readme
# GitHub → Compare & pull request → 제목·설명 작성 → Create pull request
# 짝과 서로 Files changed 탭에서 코멘트 남기기 → Merge pull request → Delete branch
git switch main && git pull
git branch -d feature-readme
```

확인 포인트

- [ ] GitHub에 노란 배너로 Compare & pull request 버튼이 나타난다
- [ ] Files changed 탭에서 무엇이 바뀌었는지 줄 단위로 보인다
- [ ] 병합 후 로컬에서 pull 하면 README.md 가 들어와 있다

> PR의 가치는 병합 자체가 아니라 **왜 이렇게 바꿨는지가 기록으로 남는 것**에 있습니다.

STEP 08 태그로 릴리즈 표시하기

특정 시점에 이름을 붙입니다.

```
git tag -a v1.0.0 -m "첫 번째 릴리즈"
git tag
git push origin v1.0.0
```

확인 포인트

- [ ] GitHub 저장소의 Tags / Releases 에 v1.0.0 이 보인다

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`Support for password authentication was removed`	비밀번호 인증 폐지	Personal Access Token 발급해 비밀번호 자리에 입력 · 또는 SSH 방식 사용
`Updates were rejected (non-fast-forward)`	원격에 내가 없는 커밋이 있음	git pull → 충돌 해결 → git push (강제 push 금지)
`src refspec main does not match any`	커밋이 하나도 없는 상태에서 push	먼저 git commit 을 한 번 하고 push
`fatal: remote origin already exists`	이미 origin이 연결됨	git remote set-url origin <새 주소>
`Permission denied (publickey)`	SSH 키 미등록	ssh-keygen 실행 후 공개키를 GitHub Settings → SSH keys 에 등록

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] fetch와 pull의 차이를 한 문장으로 설명해 보세요.
- [ ] origin/main 은 어디에 저장되어 있는 정보인가요?
- [ ] push가 거절되는 이유와 올바른 해결 순서는 무엇인가요?
- [ ] 그냥 push해도 되는데 굳이 Pull Request를 쓰는 이유는 무엇인가요?
- [ ] git push -u 의 -u 는 무슨 뜻인가요?

한 줄 정리

원격 작업은 '누가 무엇을 알고 있는가'의 문제입니다. 내 로컬이 원격을 얼마나 알고 있는지 확인하고 나서 밀어 넣으면 거절될 일이 없습니다.

다음 시간 → 4교시 — 여기서부터 Docker입니다

WORKSHEET

막힌 STEP 번호, 화면에 나온 오류 메시지, 강사에게 물어볼 것을 적어 두세요.